

TRATAMENTO DE ERROS

BUGS OU ERROS

```
private int CalcularSoma(int a, int b)
{
    int soma = a - b;
    return soma;
}
```

```
class Class1
{
    List<string> lista;

    0 references
    public Class1()
    {
        lista.Add("Item");
    }
}
```

- Ambas têm comportamento inválido
- CalcularSoma() tem um erro de lógica, mas funciona como programado
- A Class1 tem um erro do programador que irá causar um crash

ERROS NO VISUAL STUDIO

```
private void listBoxClientes_SelectedIndexChanged(object sender, EventArgs e)
{
    Cliente cliente = (Cliente)listBoxClientes.SelectedItem;
    labelClienteNome.Text = cliente.Nome;
}
```

Exception Thrown

System.NullReferenceException: 'Object reference not set to an instance of an object.'

cliente was null.

[View Details](#) | [Copy Details](#) | [Start Live Share session...](#)

▲ **Exception Settings**

☒ Break when this exception type is thrown

PREVENIR E EVITAR ERROS

```
private void listBoxClientes_SelectedIndexChanged(object sender, EventArgs e)
{
    if (listBoxClientes.SelectedItem == null)
    {
        return;
    }
    Cliente cliente = (Cliente)listBoxClientes.SelectedItem;
}
```

- Preferível antecipar do que tratar erros
- Várias estratégias para o fazer, dependendo do caso:
 - Verificar se os objetos são nulos antes de os utilizar
 - Impedir utilizador de introduzir dados vazios ou inválidos
 - Certas operações matemáticas não são possíveis
 - Nem sempre é possível realizar casts ou conversões de dados
 - ...

EXCEPTION

- Representa um comportamento inesperado que interrompe o funcionamento normal do programa/aplicação
- Ajuda a prevenir bugs e erros de utilização
- Muito importante para um código robusto
- Torna as aplicações mais fiáveis
- Exceptions não corrigem erros de programação!



EXCEPTION

- Representam erros durante a execução da aplicação

- Podem ser lançadas por:
 - Qualquer método.
- Exemplos de exceções
 - Operações matemáticas incorretas.
 - Utilização de referências não instanciadas (NullPointerException)
 - Tentativa de conversão de dados inválida
 - Utilização incorreta de ficheiros
 - Time-out em operações longas
 - ...

- Podem ser criadas e lançadas também pelo programador

TRY.. CATCH.. FINALLY..

- Define o bloco de Código que permite tratar uma exceção/erro
 - *Try* = Tenta, *Catch* = Apanha, *Finally* = Finalmente
 - Ou seja, “tenta apanhar o erro e depois faz o finalmente”.

```
try
{
    // Código a avaliar.
}
catch (SomeSpecificException ex)
{
    // Código a executar caso exista um erro (Tratamento).
}
finally
{
    // Código executado depois (é sempre executado, haja ou não exceção).
    // Opcional
}
```

TRY.. CATCH.. FINALLY..

- **TRY** – Bloco de código onde pode ocorrer o erro.
- **CATCH** – Bloco de código onde será tratado o erro.
 - Geralmente utilizado para apresentar uma mensagem.
- **FINALLY (opcional)** – Bloco que é sempre executado independentemente de ser ou não lançada uma exceção.
 - Geralmente utilizado para libertar algum recurso independentemente do erro ocorrer.
- **THROW** – Utilizado quando o programador pretende que seja lançada uma exceção específica. Controlo de erros com exceções.

```
if (tam <= 0)
```

```
    throw new Exception("A quantidade de caracteres a ler deve ser maior do que 0.");
```


MÚLTIPLAS EXCEPTIONS

- Num bloco de código podemos tratar várias exceções.
- Tal é implementado através da colocação de vários *catch's*.
- Os *catch's* devem ser incluídos iniciando pelo mais detalhado.

```
try {  
    //código  
    MessageBox.Show("Dinheiro Libertado");  
} catch (SaldoInsuficienteException ex) {  
    MessageBox.Show("Saldo insuficiente");  
} catch (ArgumentException ex) {  
    MessageBox.Show("Não é possível libertar um valor negativo");  
}
```

HERANÇA EM EXCEPTIONS

- Exceptions são classes, permitindo que sejam herdadas.
- Permite a criação de Exceptions customizadas a cada necessidade

```
class CustomException : Exception
{
    int code;

    public CustomException(int code, string message) : base(message)
    {
        this.code = code;
    }
}
```

UTILIZAÇÃO DE EXCEPTIONS

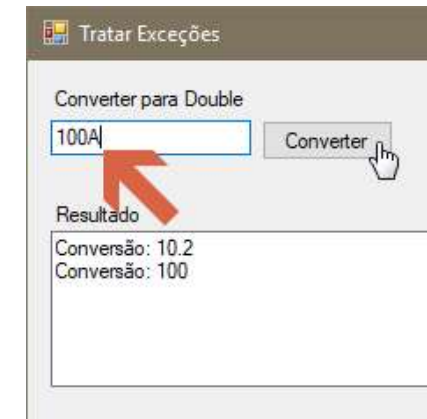
- Quando não temos tratamento de exceções a execução do programa é interrompida quando acontece um erro:

```
1 reference
private void btConverter_Click(object sender, EventArgs e) {
    Double valor = Convert.ToDouble( tbValor.Text );
    tbResultado.AppendText( "Conversão: " + valor + Environment.NewLine );
}
```

! FormatException was unhandled

An unhandled exception of type 'System.FormatException' occurred in mscorlib.dll

Additional information: Input string was not in a correct format.

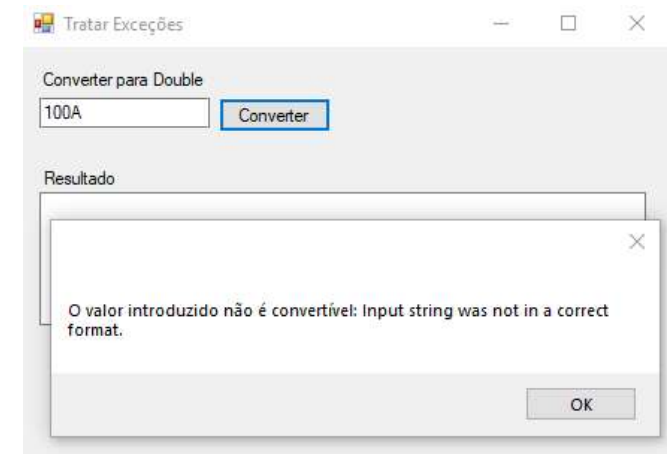


- Neste caso, o que ocorreu foi uma conversão de uma *String* para *Double*. Mas “100A” não é convertível para *Double*. Neste caso é lançada uma exceção que pode ser apanhada...
- **Vamos tratar a exceção**

PROTEGER O CÓDIGO COM EXCEPTIONS

- Vamos aplicar um try ... catch ao bloco de código que faz a conversão:

```
try {  
    Double valor = Convert.ToDouble( tbValor.Text );  
    tbResultado.AppendText( "Conversão: " +  
        valor + Environment.NewLine );  
} catch( FormatException fEx ){  
    MessageBox.Show( "O valor introduzido não é convertível: " +  
        fEx.Message );  
} catch (Exception ex) {  
    MessageBox.Show("Ocorreu um erro ao realizar a conversão: " +  
        ex.Message);  
}
```



EXCEPTIONS COMUNS

Tipo de exceção	Descrição	Exemplo
<u>Exception</u>	A classe base de todas as exceções.	Nenhum (use classes derivadas dessa exceção).
<u>IndexOutOfRangeException</u>	Gerada em tempo de execução somente quando uma matriz é indexada incorretamente.	Indexar uma matriz fora do intervalo válido: <code>arr[arr.Length+1]</code>
<u>NullReferenceException</u>	Gerada pelo tempo de execução somente quando um objeto nulo é referenciado.	<code>object o = null;</code> <code>o.ToString();</code>
<u>InvalidOperationException</u>	Gerada por métodos quando em um estado inválido.	Chamar <code>Enumerator.MoveNext()</code> após a remoção de um item da coleção subjacente.
<u>ArgumentException</u>	A classe base para todas as exceções de argumento.	Nenhum (use classes derivada dessa exceção).
<u>ArgumentNullException</u>	Gerada por métodos que não permitem que um argumento seja nulo.	<code>String s = null;</code> <code>"Calculate".IndexOf(s);</code>
<u>ArgumentOutOfRangeException</u>	Gerada por métodos que verificam se os argumentos estão em um determinado intervalo.	<code>String s = "string";</code> <code>s.Substring(s.Length+1);</code>

FONTES

<https://docs.microsoft.com/pt-br/dotnet/standard/exceptions/>

<https://docs.microsoft.com/en-us/dotnet/csharp/programming-guide/exceptions/>

https://www.tutorialspoint.com/csharp/csharp_exception_handling.htm

<https://www.tutorialsteacher.com/csharp/csharp-exception>